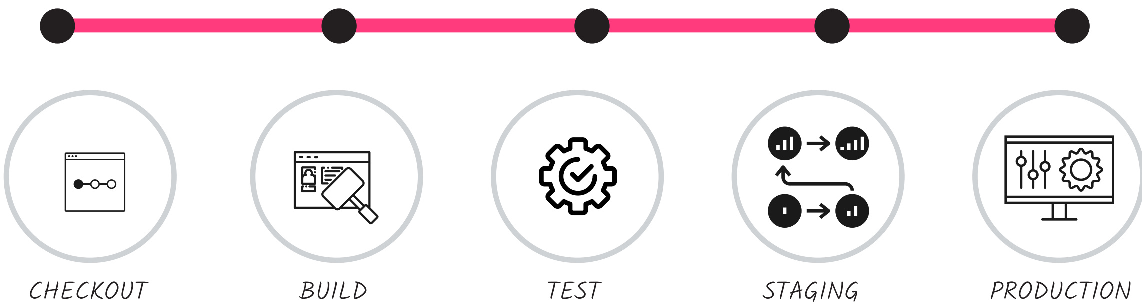
Working with Jenkins Pipelines

Jenkins is an open source tool that can integrate with the tools used in application life cycle management to automate the entire software development process. In this article, we will discuss different types of pipelines that we can create using Jenkins.



Jenkins installation is easy with different platform-agnostic options such as Java package (.*war*), Docker, FreeBSD, Gentoo, Arch Linux, macOS, Red Hat/Fedora/CentOS, Ubuntu/Debian, OpenBSD, OpenSuse, OpenIndiana Hipster, and Windows. Arguably, it supports all languages and tools as it provides plugin-based architecture. In case plugins are not available, CLI or command execution helps to set up continuous practices.

The installation and configuration processes in Jenkins are different since the release of Jenkins 2.0. Suggested plugins can be installed now. Jenkins 2.0 and later versions have built-in support for pipeline as code. Jenkins 2.x has improved usability and is fully backward-compatible. Let's understand the traditional pipeline using the build pipeline plugin.

Build pipeline

Pipeline as code is preferred over the traditional approach of using upstream and downstream jobs for a number of reasons. In the traditional approach:

- Each task has a corresponding unique build job or project; hence, there are too many jobs to create and manage.
- A business unit or organisation manages multiple projects.
- Build pipeline configuration is in Jenkins itself, and in case of failure it is a rework.
- It is difficult to track changes in the pipeline created using the build pipeline.